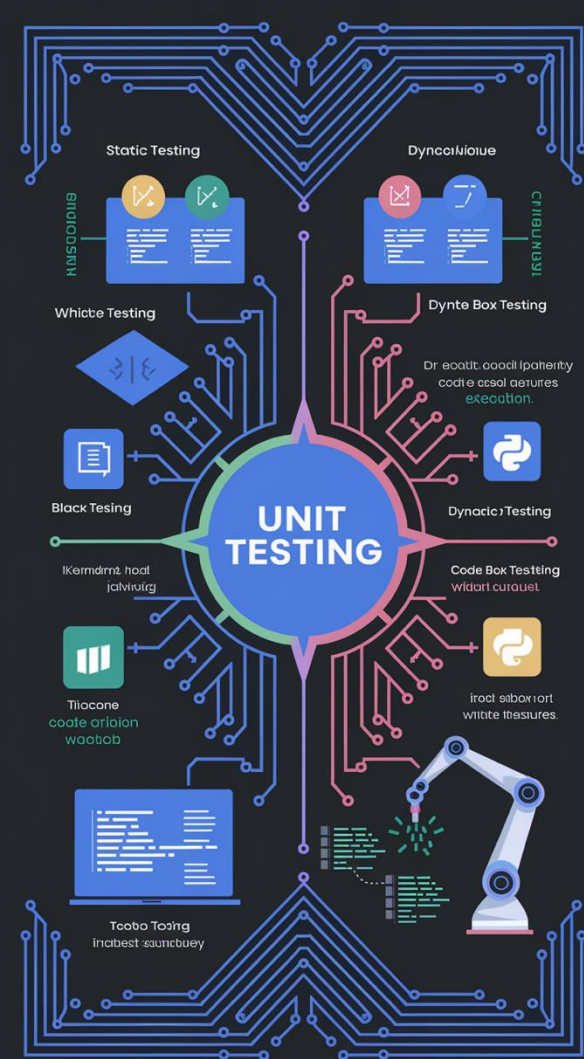


Pruebas de Software: Pruebas Unitarias

Semana 10

Ingeniería de Software 2

2026-1





Logro de la sesión



Los estudiantes son capaces de comprender y aplicar los conceptos fundamentales de las pruebas unitarias, diferenciando entre pruebas estáticas y dinámicas. Utilizan técnicas de pruebas de caja blanca y caja negra para diseñar y ejecutar casos de prueba efectivos, asegurando la calidad del software.

- ¿Alguna vez te has preguntado cómo las aplicaciones que usas a diario, como WhatsApp o Netflix, funcionan sin errores críticos la mayor parte del tiempo?
- ¿Qué pasaría si un error en el software de un banco duplicara o eliminara transacciones? ¿Cómo podrías evitar que algo así ocurra?
- ¿Te gustaría ser el desarrollador que entrega un código confiable y de alta calidad, evitando problemas en producción y ganando la confianza de tu equipo?
- ¿Sabías que las pruebas unitarias no solo detectan errores, sino que también te ayudan a escribir mejor código y a ahorrar tiempo en el futuro?



Contenidos

- Concepto de pruebas unitarias.
- Pruebas unitarias estáticas y dinámicas.
- Pruebas de Caja Blanca.
- Pruebas de Caja Negra.
- Herramientas para pruebas.
- Automatización de pruebas unitarias.

Pruebas Unitarias

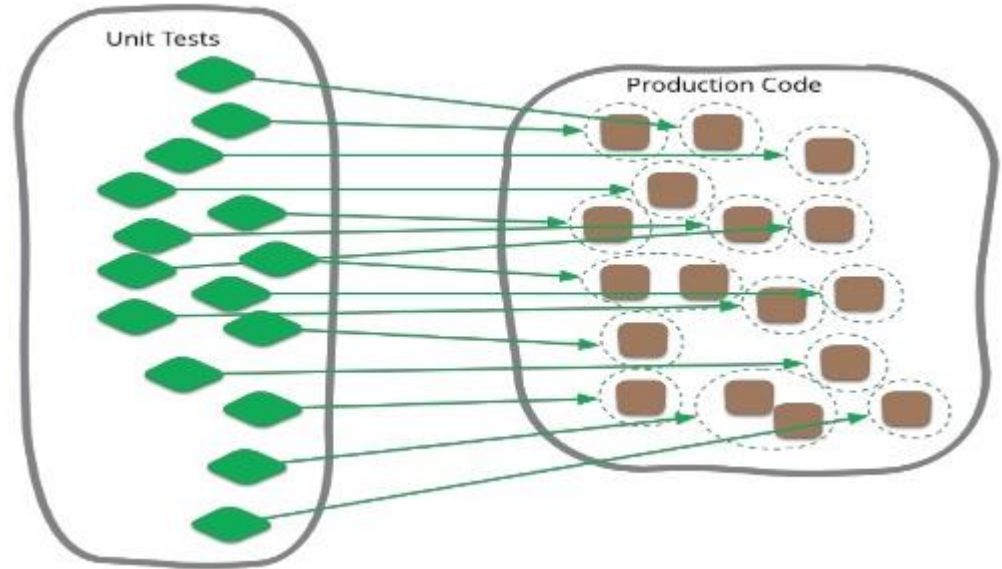
Las pruebas unitarias son un tipo de prueba de software que se centra en verificar el correcto funcionamiento de las unidades más pequeñas de código, como funciones, métodos o clases, de forma aislada.

Su objetivo principal es garantizar que cada unidad de código funcione según lo esperado, detectando errores en etapas tempranas del desarrollo. Estas pruebas suelen ser realizadas por los desarrolladores y son fundamentales para mantener la calidad del software.

- Cada pieza de código funcional (clase o método) será susceptible de ser probada con pruebas unitarias.
- Estas son pruebas automáticas (código) que se encargan de simular la interacción de entidades externas con la pieza de código (desde ahora componente).

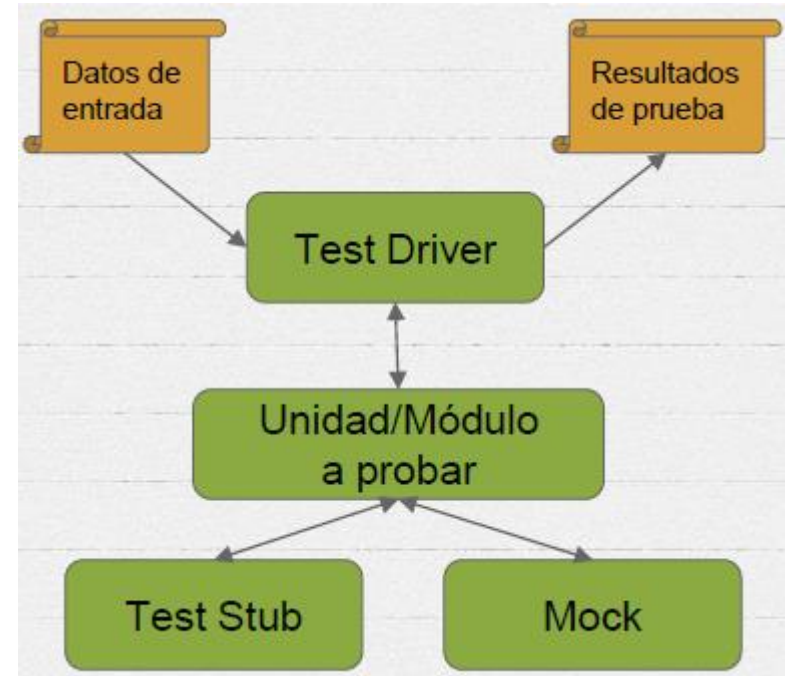
TIPOS DE PRUEBAS

- **Pruebas de Unidad (Unitarias)**
 - Se debe probar el objeto por completo.
 - Todos los métodos
 - Todos las salidas
 - Todas las entradas
 - Todos los estados
 - Todas las herencias
 - Se pueden usar las técnicas de **clases equivalentes**.



TIPOS DE PRUEBAS

- Pruebas de Unidad (Unitarias)
 - **Test Driver (Manejador de prueba):** Objeto que llama al módulo a probar.
 - **Test Stub:** Objeto (que el módulo a probar llama) que siempre devuelve un resultado estático.
 - **Mock Object:** Objeto (que el módulo a probar llama) que devuelve resultados, sin que esté implementado.



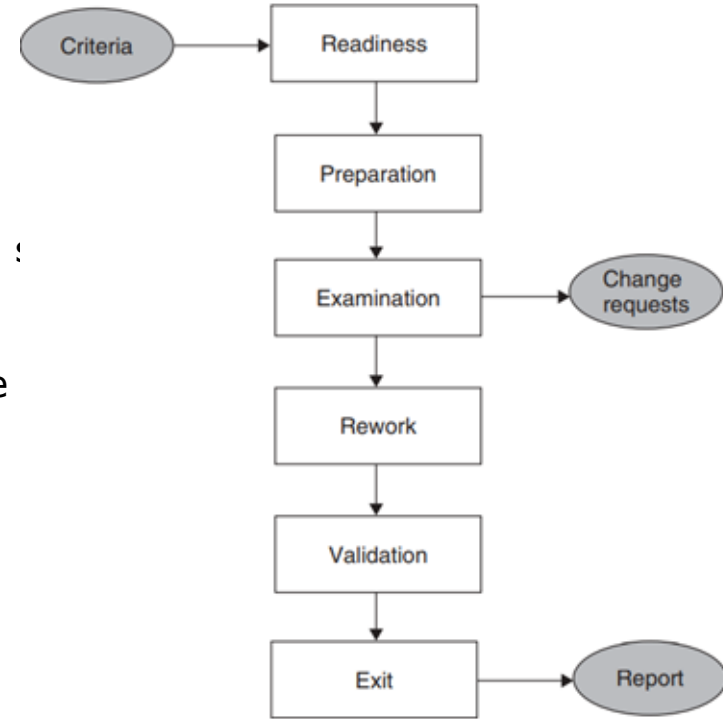
Pruebas Unitarias Estáticas y Dinámicas

Prueba unitaria estática

- Se llevan a cabo en una **fase de inspección** y corrección donde el producto no necesita estar en forma final. Las pruebas estáticas no ejecutan el código, sino que se centran en analizarlo para identificar errores, inconsistencias o problemas de calidad.

Ejemplos incluyen:

- Revisión de código (code review).
- Análisis estático mediante herramientas como SonarQube o ESLint.
- Verificación de estándares de codificación.



Code review



Prueba unitaria dinámicas

Las pruebas dinámicas implican la **ejecución del código** para verificar su comportamiento. En este caso, se escriben y ejecutan casos de prueba específicos para comprobar que las unidades de código producen los resultados esperados. Estas pruebas suelen automatizarse con frameworks como JUnit, NUnit o pytest.

Se diferencia de la ejecución ordinaria en lo siguiente:

1. Una unidad bajo prueba **se saca de su entorno** de ejecución real.
2. El entorno de ejecución real **se emula** escribiendo más código para que la unidad y el entorno emulado se puedan compilar juntos.
3. La unidad bajo prueba y el código adicional se ejecuta **con entradas seleccionadas previamente**. El resultado de una ejecución de este tipo se recopila de diversas formas y se compara con el resultado esperado. Cualquier diferencia entre el resultado real y el esperado implica una falla y la falla está en el código.



Marco de pruebas unitarias xUnit

xUnit es una familia de frameworks de pruebas unitarias ampliamente utilizada en el desarrollo de software. Su nombre proviene de la convención de nombrar frameworks de pruebas basados en el diseño original de JUnit (para Java), y la "x" representa la variedad de lenguajes y plataformas en los que se implementa. Estos frameworks están diseñados para facilitar la escritura, ejecución y automatización de pruebas unitarias.

La arquitectura xUnit tiene estos componentes comunes:

- Caso de prueba/conjuntos de pruebas
- Accesorio de prueba
- Corredor de prueba
- Formateador de resultados de prueba
- Afirmaciones:

Características Principales de xUnit

- **Estructura estándar:** Proporciona una estructura común para escribir pruebas unitarias, lo que facilita la adopción en diferentes lenguajes y plataformas.
- **Automatización:** Permite ejecutar pruebas automáticamente, integrándose con herramientas de integración continua (CI/CD).
- **Aislamiento:** Las pruebas se ejecutan de forma independiente, asegurando que los resultados de una prueba no afecten a las demás.
- **Assertions:** Incluye métodos para verificar que los resultados de las pruebas sean los esperados.
- **Configuración y limpieza:** Proporciona métodos para preparar el entorno antes de ejecutar las pruebas (setup) y limpiarlo después (teardown).



Ejemplos de Frameworks xUnit por Lenguaje

Algunos de los frameworks más populares basados en xUnit incluyen:

- **JUnit:** Para Java, es uno de los frameworks más antiguos y ampliamente utilizados.
- **NUnit:** Para .NET, utilizado en aplicaciones C# y VB.NET.
- **pytest:** Aunque no sigue estrictamente la nomenclatura "xUnit", es un framework popular para Python que adopta principios similares.
- **PHPUnit:** Para PHP, diseñado para pruebas unitarias en aplicaciones web.
- **CppUnit:** Para C++, basado en el diseño de JUnit.
- **Node.js Test:** Pruebas básicas en NodeJS.
- **xUnit.net:** Una implementación moderna para .NET, que mejora y simplifica el diseño de NUnit.



Ciclo de Vida de una Prueba en xUnit

El ciclo de vida típico de una prueba en xUnit incluye:

- **Setup:** Configuración inicial del entorno de prueba (por ejemplo, inicializar objetos o datos).
- **Ejecución:** Se ejecuta el código que se desea probar.
- **Assertion:** Se verifican los resultados esperados mediante afirmaciones.
- **Teardown:** Limpieza del entorno de prueba (por ejemplo, liberar recursos o restablecer estados).

MAVEN

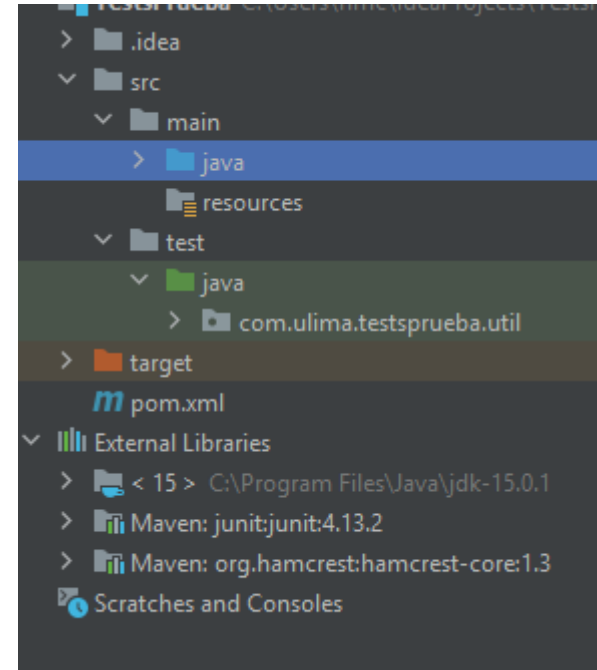
Maven es una herramienta de software para la gestión y construcción de proyectos **Java**

- Tiene un modelo de configuración de construcción simple, basado en un formato **XML**
- Maven utiliza un ***Project Object Model (POM)*** para describir el proyecto de software a construir, sus dependencias de otros módulos y componentes externos, y el orden de construcción de los elementos

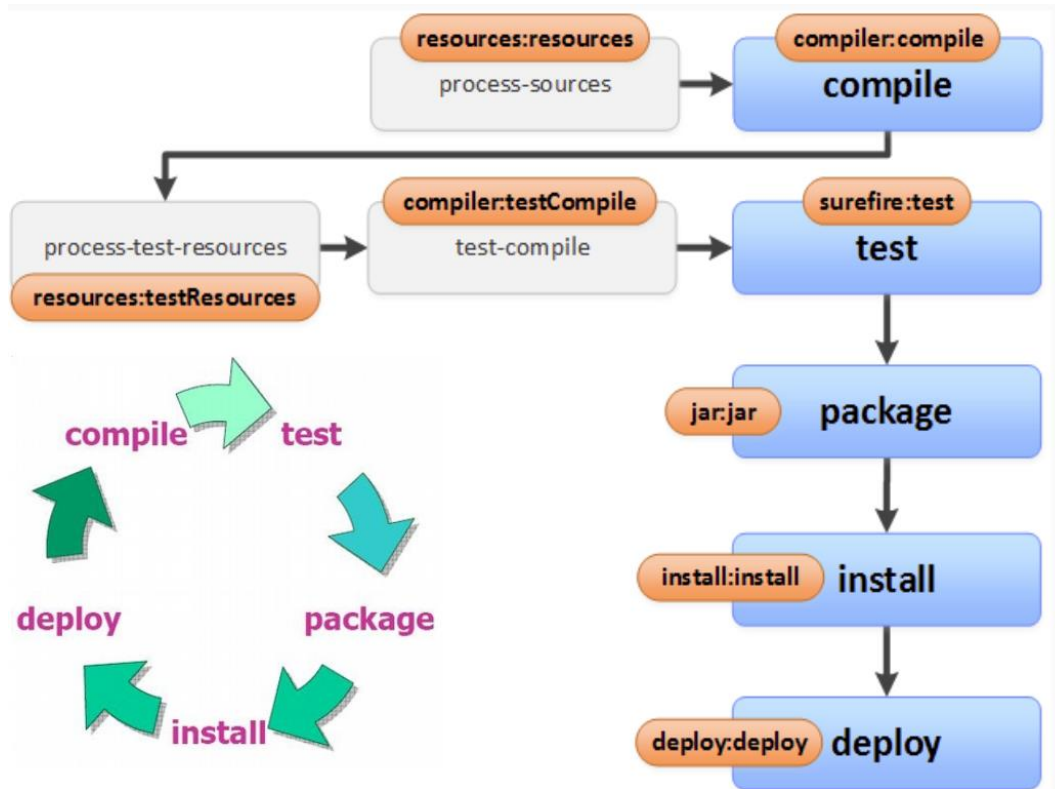
MAVEN

La estructura :

- **src/main/java** : contiene las clases java.
- **src/main/resources** : contiene XMLs, .properties, etc.
- **src/test/java** : contiene las clases java usadas en los tests.
- **src/test/resources** : contiene XMLs, .properties, etc. usados en los tests.



Ciclo de vida del proyecto Maven



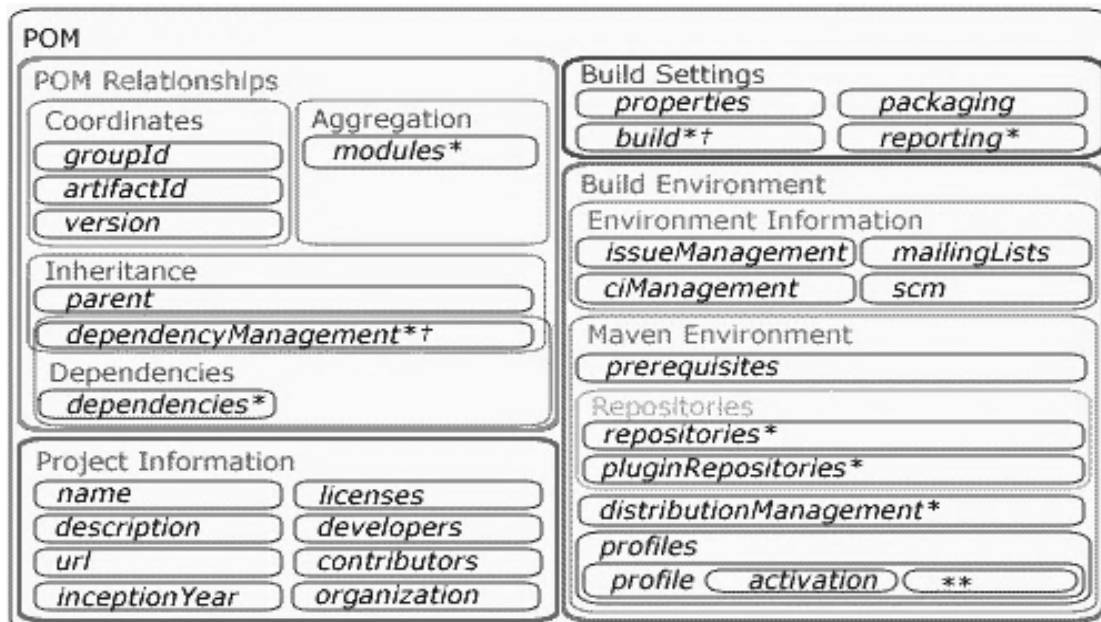
Estructura de un proyecto- Maven

- **groupId:** Aquí se pone el nombre de la empresa u organización, todos los proyectos con ese groupId pertenecen a una sola empresa.
- **artifactId:** Es el nombre del proyecto.
- **version:** Número de versión del proyecto.
- **package:** Paquete base donde irá el código fuente

Opción	Ejemplo
groupId	com.empresa
artifactId	proyecto
version	1.3
package	com.empresa.paquete

Maven

- Esquema de la estructura de un fichero pom.xml



```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0"
  <modelVersion>4.0.0</modelVersion>

  <groupId>com.ulima e</groupId>
  <artifactId>TestsPrueba</artifactId>
  <version>1.0-SNAPSHOT</version>

  <properties>
    <maven.compiler.target>1.8</maven.compiler.target>
    <maven.compiler.source>1.8</maven.compiler.source>
  </properties>

</project>
```

JUnit

JUnit (@ <http://junit.org/>) es un *marco de pruebas unitarias* de Java de código abierto fue creado en **1997** por **Erich Gamma** y **Kent Beck**. Es uno de los primeros frameworks de pruebas unitarias y se desarrolló como parte del movimiento de **Desarrollo Guiado por Pruebas (TDD, Test-Driven Development)**, promovido por Kent Beck.

JUnit fue diseñado para facilitar la escritura y ejecución de pruebas unitarias en Java, y su éxito inspiró la creación de otros frameworks similares en diferentes lenguajes.



Las dependencias que necesitamos añadir al pom.xml son:

```
1<dependency>
2  <groupId>junit</groupId>
3  <artifactId>junit</artifactId>
4  <version>4.8.2</version>
5  <scope>test</scope>
6</dependency>
7<dependency>
8  <groupId>org.springframework</groupId>
9  <artifactId>spring-test</artifactId>
10 <version>3.2.4.RELEASE</version>
11 <scope>test</scope>
12</dependency>
13<dependency>
14 <groupId>org.mockito</groupId>
15 <artifactId>mockito-core</artifactId>
16 <version>1.8.5</version>
17 <scope>test</scope>
18</dependency>
```

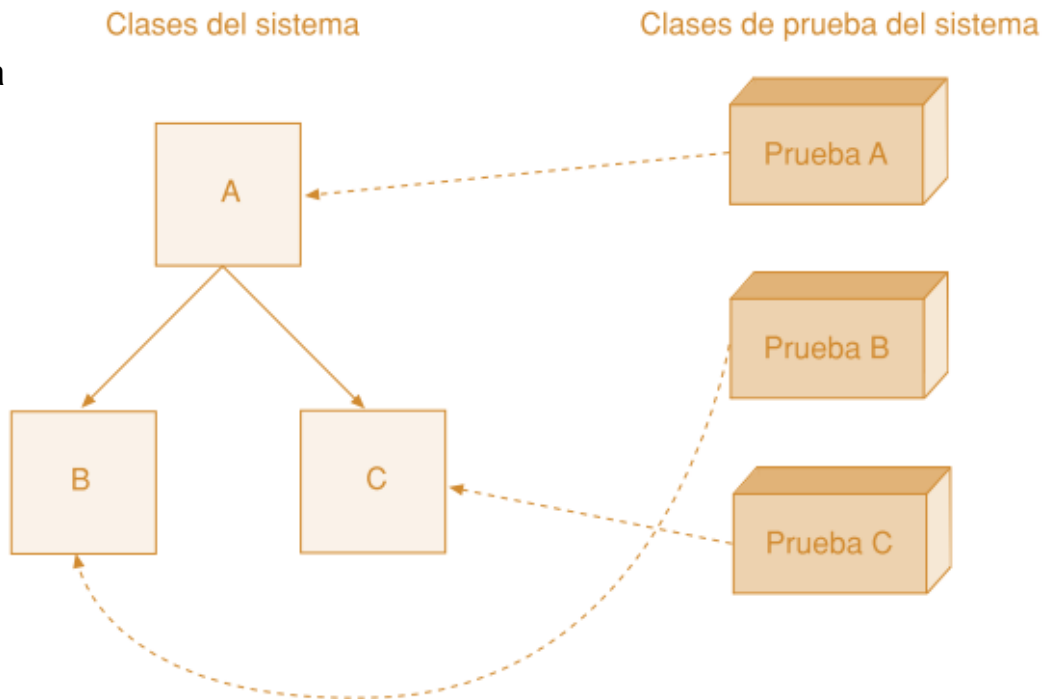


JUnit es un paquete Java
para automatizar las pruebas
de clases Java.
<http://junit.org/>



Relaciones entre las clases de un sistema y las correspondientes clases de pruebas unitarias.

Cuando se trabaja con JUnit se ha de crear una clase para cada clase que se desee probar. Dicha clase, será llamada en adelante “**clase de pruebas**” y contendrá todo el código necesario para la ejecución de los diferentes casos de prueba definidos para la clase a probar también conocida como “**clase objetivo**”.



Junit - @Test

```
import static org.junit.Assert.*;
import org.junit.Test;

public class Ejemplo {

    @Test
    public void test() {
        // prueba
    }
}
```

Los casos de prueba son métodos que

- ☐ no devuelven nada: void
- ☐ están etiquetados como @Test
- ☐ no tienen argumentos ()
- ☐ contienen aserciones

Junit - Assert

assertEquals (X esperado, Xreal)	compara un resultado esperado con el resultado obtenido, determinando que la prueba pasa si son iguales, y que la prueba falla si son diferentes. Usa el método equals().
assertSame(X esperado, X real)	Ídem; pero usa == para determinar si es el objeto esperado.
assertFalse (boolean resultado)	verifica que el resultado es FALSE
assertTrue (boolean resultado)	verifica que el resultado es TRUE
assertNull (Object resultado)	verifica que el resultado es "null"
assertNotNull (Object resultado)	verifica que el resultado no es "null"
assertThrows(IllegalArgumentException.class, () -> servicio.procesar(null));	Verifica excepciones
assertAll(() -> assertEquals(1, x), () -> assertTrue(y > 0));	Varias verificaciones anidadas.



JUnit 5

Anotaciones del ciclo de vida

Anotación	Descripción
<code>@Test</code>	Marca un método como test
<code>@BeforeEach</code>	Ejecuta antes de cada test
<code>@AfterEach</code>	Ejecuta después de cada test
<code>@BeforeAll</code>	Una vez antes de todos los tests
<code>@AfterAll</code>	Una vez después de todos los tests
<code>@Disabled</code>	Deshabilita temporalmente un test

Aserciones básicas

```
import static org.junit.jupiter.api.Assertions.*;

@Test
void ejemploAserciones() {
    assertEquals(4, 2 + 2);
    assertTrue(lista.isEmpty());
    assertNull(resultado);
    assertThrows(
        IllegalArgumentException.class,
        () -> servicio.procesar(null)
    );
    assertAll(
        () -> assertEquals(1, x),
        () -> assertTrue(y > 0)
    );
}
```



Creación de una clase de pruebas con JUnit

1. Definición de la Clase de Prueba
2. Inicialización y Liberación de Recursos:
3. Uso de Métodos assert:
4. Ejecución de las Pruebas:



Creación de una clase de pruebas con JUnit

1. Definición de la Clase de Prueba :

- La clase de prueba debe heredar de la clase `TestCase` del paquete `junit.framework`. Esto es necesario porque `TestCase` proporciona los mecanismos y métodos necesarios para realizar las pruebas, como los métodos `assert` (por ejemplo, `assertNull`, `assertEquals`, `assertTrue`, etc.).
- Por convención, el nombre de la clase de prueba debe ser el nombre de la clase a probar seguido del sufijo `Test`. Ejemplo: Si la clase a probar es `Log`, la clase de prueba se llamará `LogTest`.
- Es importante importar el paquete `junit.framework` o las clases específicas que se utilizarán.



Creación de una clase de pruebas con JUnit

2. Inicialización y Liberación de Recursos:

- JUnit permite definir métodos especiales para preparar y limpiar los recursos necesarios para las pruebas:
 - @Before: Método que inicializa los recursos antes de ejecutar cada prueba.
 - @After: Método que libera los recursos después de ejecutar cada prueba.
- Estos métodos aseguran que cada prueba se ejecute en un entorno limpio y controlado.



Creación de una clase de pruebas con JUnit

3. Uso de Métodos assert:

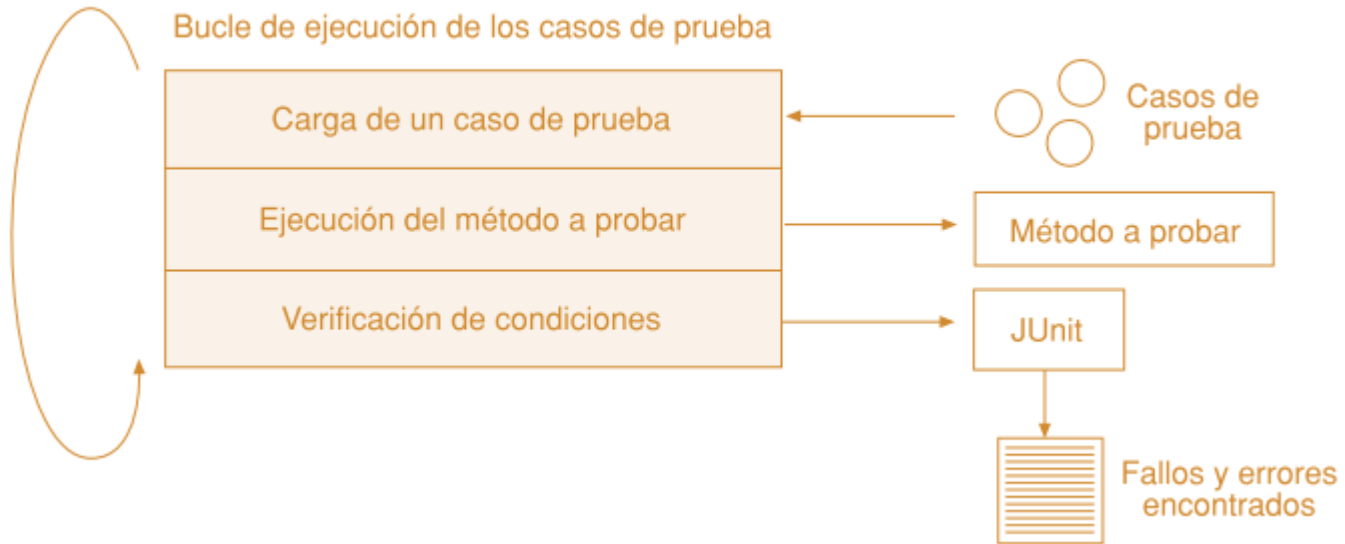
- Los métodos assert se utilizan para verificar condiciones en las pruebas. Si una condición no se cumple, se lanza un error de tipo `AssertionFailedError`, indicando que la prueba ha fallado

4. Ejecución de las Pruebas:

- Estos métodos aseguran que cada prueba se ejecute en un entorno limpio y controlado.

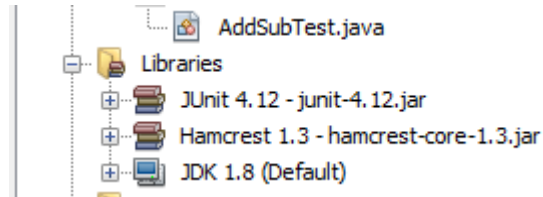


Esquema de ejecución de un método de prueba

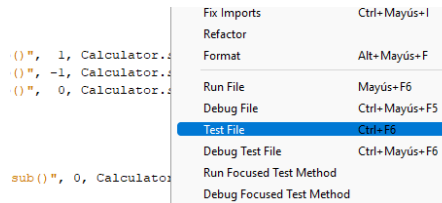


Ejemplo – Caso de prueba

- Cree un nuevo proyecto Java de llamado " JUnitTest".
- Cree una nueva clase llamada " Calculator"
- Adicione un JUnit Test y Cree el primer caso de prueba llamado " AddSubTest"
- Agregue las librerías



- Ejecute las pruebas

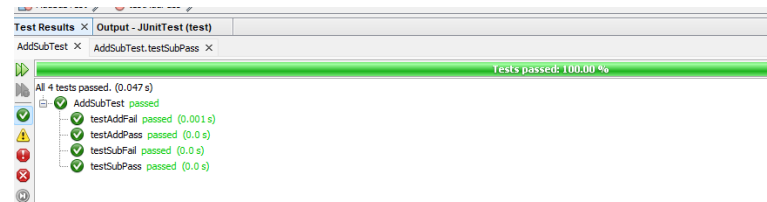


Calculator.java

```
4 public class Calculator {
5     public static int add(int number1, int number2) {
6         return number1 + number2;
7     }
8
9     public static int sub(int number1, int number2) {
10        return number1 - number2;
11    }
12
13    public static int mul(int number1, int number2) {
14        return number1 * number2;
15    }
16
17    public static int divInt(int number1, int number2) {
18        if (number2 == 0) {
19            throw new IllegalArgumentException("No se puede dividir por 0!");
20        }
21        return number1 / number2;
22    }
23
24    public static double divReal(int number1, int number2) {
25        if (number2 == 0) {
26            throw new IllegalArgumentException("No se puede dividir por 0!");
27        }
28        return (double) number1 / number2;
29    }
30 }
31
```

AddSubTest.java

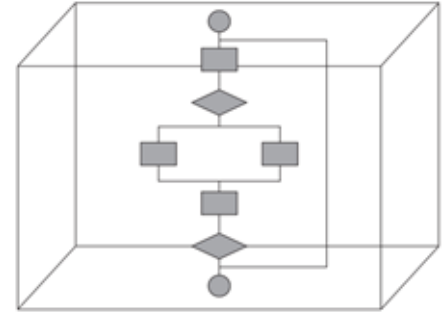
```
1
2 import junittest.Calculator;
3 import static org.junit.Assert.*;
4 import org.junit.Test;
5
6 public class AddSubTest {
7     @Test
8     public void testAddPass() {
9         assertEquals("error en add()", 3, Calculator.add(1, 2));
10        assertEquals("error en add()", -3, Calculator.add(-1, -2));
11        assertEquals("error en add()", 9, Calculator.add(9, 0));
12    }
13
14    @Test
15    public void testAddFail() {
16        assertEquals("error en add()", 0, Calculator.add(1, 2));
17    }
18
19    @Test
20    public void testSubPass() {
21        assertEquals("error en sub()", 1, Calculator.sub(2, 1));
22        assertEquals("error en sub()", -1, Calculator.sub(-2, -1));
23        assertEquals("error en sub()", 0, Calculator.sub(2, 2));
24    }
25
26    @Test
27    public void testSubFail() {
28        assertEquals("error en sub()", 0, Calculator.sub(2, 1));
29    }
30
31 }
```



Pruebas de caja blanca VS pruebas de caja negra

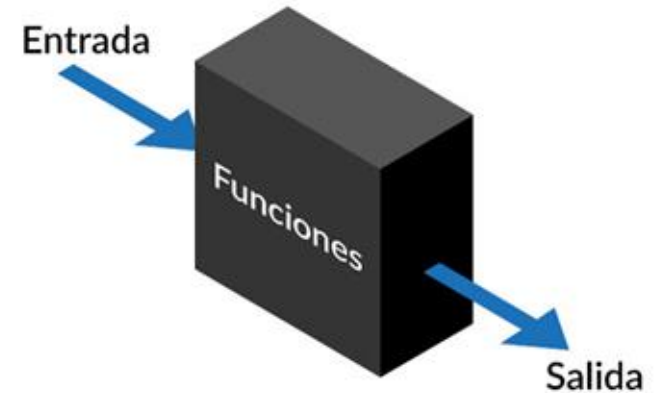
Pruebas de caja blanca

- Pruebas de caja blanca (White-Box Testing). Son pruebas estructurales. Conociendo el código y siguiendo su estructura lógica, se pueden diseñar pruebas destinadas a comprobar que el código hace correctamente lo que el diseño de bajo nivel indica y otras que demuestren que no se comporta adecuadamente ante determinadas situaciones.
- Ejemplos típicos de ello son las pruebas unitarias. Se centran en lo que hay codificado o diseñado a bajo nivel por lo que no es necesario conocer la especificación de requisitos, que por otra parte será difícil de relacionar con partes diseñadas a muy bajo nivel.

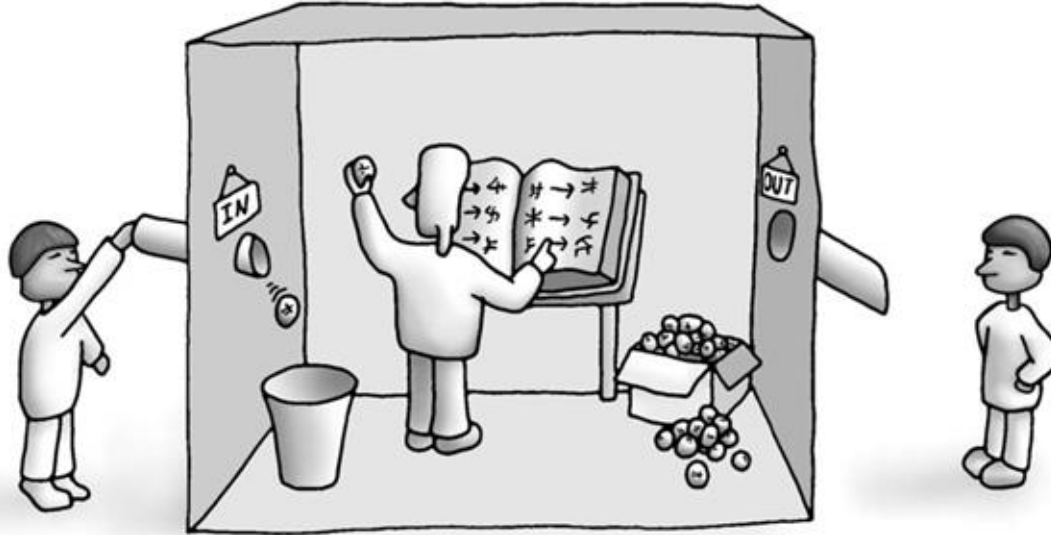


Pruebas de caja negra

- Las pruebas de caja negra (Black-Box Testing) son pruebas funcionales. Se parte de los requisitos funcionales, a muy alto nivel, para diseñar pruebas que se aplican sobre el sistema sin necesidad de conocer como está construido por dentro (Caja negra).
- Las pruebas se aplican sobre el sistema empleando un determinado conjunto de datos de entrada y observando las salidas que se producen para determinar si la función se está desempeñando correctamente por el sistema bajo prueba. Las herramientas básicas son observar la funcionalidad y contrastar con la especificación.



Prueba de Caja Blanca VS Caja Negra



jolyon.co.uk

Pruebas de Caja Blanca: Estáticas y Dinámicas

- **Estáticas:** En el caso de las pruebas de caja blanca, se pueden realizar análisis estáticos del código sin ejecutarlo. Por ejemplo:
 - Uso de **grafos de control de flujo** para analizar caminos posibles en el código.
 - Revisión manual del código para identificar errores lógicos o problemas de implementación.
 - Análisis estático con herramientas como SonarQube o Checkstyle para detectar problemas de calidad o errores potenciales.
- **Dinámicas:** Cuando se ejecuta el código para verificar su comportamiento interno, como la cobertura de ramas, condiciones o caminos específicos, las pruebas de caja blanca se consideran dinámicas.



Pruebas de Caja Negra: Estáticas y Dinámicas

- **Estáticas:** En las pruebas de caja negra, se pueden realizar análisis sin ejecutar el código, como:
 - Aplicar técnicas como **partición de equivalencia** o análisis de valores límite para diseñar casos de prueba basados en los requisitos.
 - Revisar especificaciones, diagramas o documentación para identificar posibles errores en la lógica funcional.
- **Dinámicas:** Cuando se ejecuta el software para verificar que las entradas producen las salidas esperadas, las pruebas de caja negra se consideran dinámicas.



Pruebas de caja blanca

- La prueba de la caja blanca usa la estructura de control del diseño procedural para derivar los casos de prueba.
- Idea: no es posible probar todos los caminos de ejecución distintos, pero sí confeccionar casos de prueba que garanticen que se verifican todos los caminos de ejecución llamados independientes.
- Verificaciones para cada camino independiente:
 - Probar sus dos facetas desde el punto de vista lógico, es decir, verdadera y falsa.
 - Ejecutar todos los bucles en sus límites operacionales
 - Ejercitar las estructuras internas de datos.

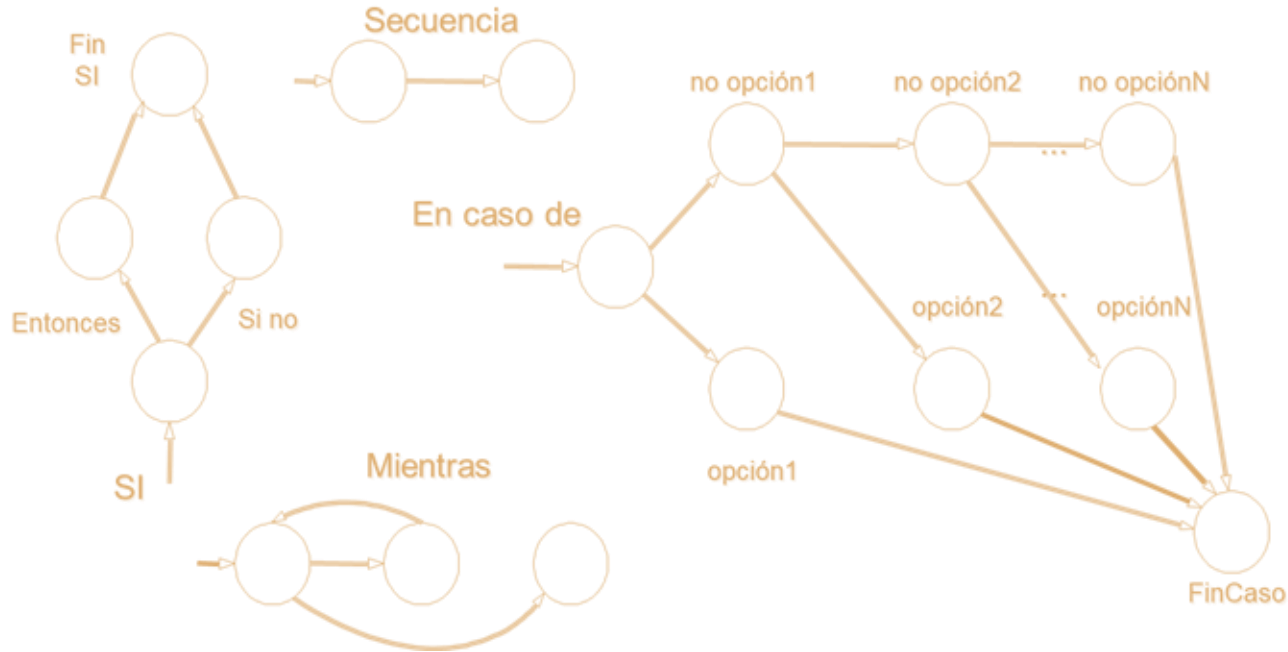
Prueba del camino básico

- La **prueba del camino básico** es una técnica de prueba de la caja blanca propuesta por Tom McCabe.
- La idea es **derivar** casos de prueba a partir de un conjunto dado de **caminos independientes** por los cuales puede circular el flujo de control.
- **Camino independiente** es aquel que introduce por lo menos una sentencia de procesamiento (o condición) que no estaba considerada en el conjunto de caminos independientes calculados hasta ese momento.
- Para obtener el conjunto un conjunto de caminos independientes se construirá el Grafo de Flujo asociado y se calculará su Complejidad Ciclomática.

Prueba del camino básico: Grafos de Flujo

- El flujo de control de un programa puede representarse por un grafo de flujo.
- Cada nodo del grafo corresponde a una o más sentencias de código fuente
- Cada nodo que representa una condición se denomina nodo predicado.
- Cualquier representación del diseño procedural se puede traducir a un grafo de flujo.
- Un camino independiente en el grafo es el que incluye alguna **arista nueva**, es decir, que no estaba presente en los caminos definidos previamente.

Prueba del camino básico: Grafos de Flujo



Prueba del camino básico: Grafos de Flujo

- Si se diseñan casos de prueba que cubran los caminos básicos, se garantiza la ejecución de cada sentencia al menos una vez y la prueba de cada condición en sus dos posibilidades lógicas (verdadera y falsa).
- El conjunto básico para un grafo dado puede no ser único, depende del orden en que se van definiendo los caminos.
- Cuando aparecen condiciones lógicas compuestas la confección del grafo es más compleja.

Prueba del camino básico: Grafos de Flujo

- Ejemplo:

Si a O b

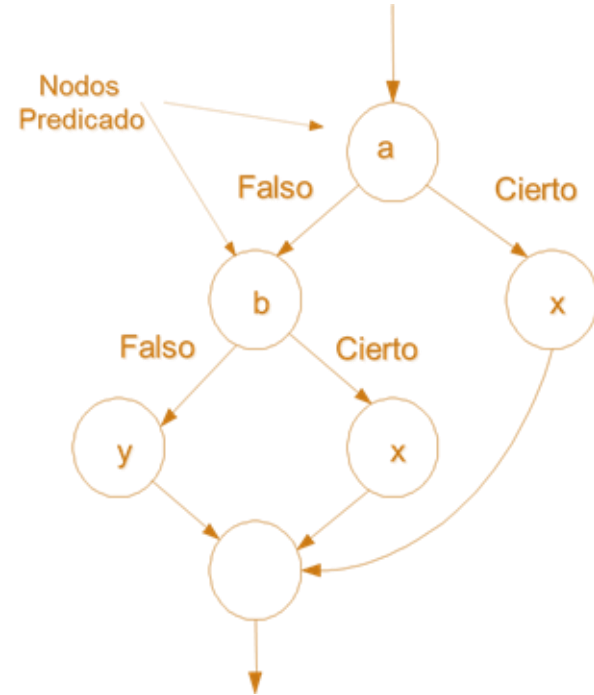
Entonces

hacer x

Si No

hacer y

FinSi

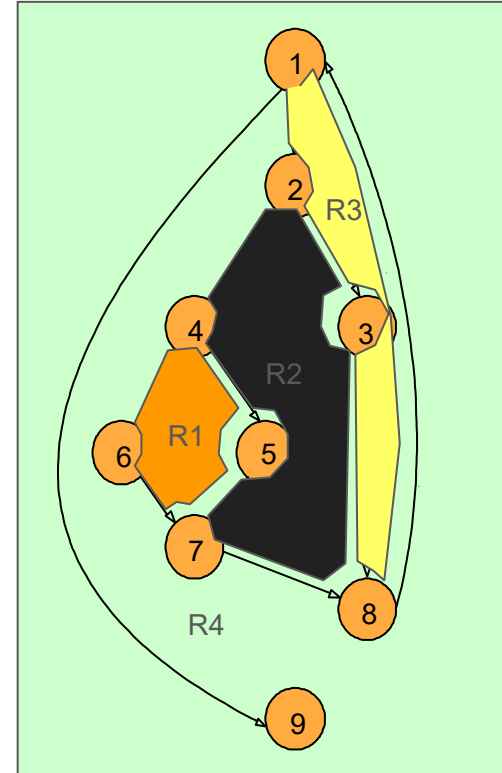


Prueba del camino básico: Complejidad Ciclomática

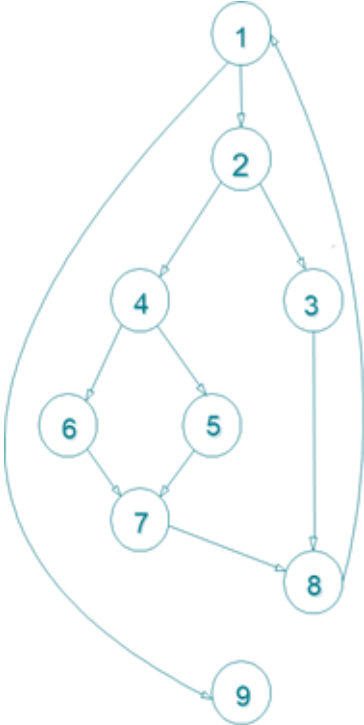
- Complejidad ciclomática de un grafo de flujo, $V(G)$, indica el número máximo de caminos independientes en el grafo.
- Puede calcularse de tres formas alternativas:
 - El **número de regiones** en que el grafo de flujo divide el plano.
 - $V(G) = A - N + 2$, donde A es el número de aristas y N es el número de nodos.
 - $V(G) = P + 1$, donde P es el número de nodos predicado.

Prueba del camino básico: Complejidad Ciclomática

- $V(G) = 4$
 - El grafo de la figura delimita cuatro regiones.
 - $11 \text{ aristas} - 9 \text{ nodos} + 2 = 4$
 - $3 \text{ nodos prediado} + 1 = 4$



Prueba del camino básico: Complejidad ciclomática



El conjunto de caminos independientes del grafo será 4.

Camino 1: 1-9

Camino 2: 1-2-4-5-7-8-1-9

Camino 3: 1-2-4-6-7-8-1-9

Camino 4: 1-2-3-8-1-9

Cualquier otro camino no será un camino independiente, p.e.,

1-2-4-5-7-8-1-2-3-8-1-2-4-6-7-8-1-9

ya que es simplemente una combinación de caminos ya especificados

Los cuatro caminos anteriores constituyen un conjunto básico para el grafo

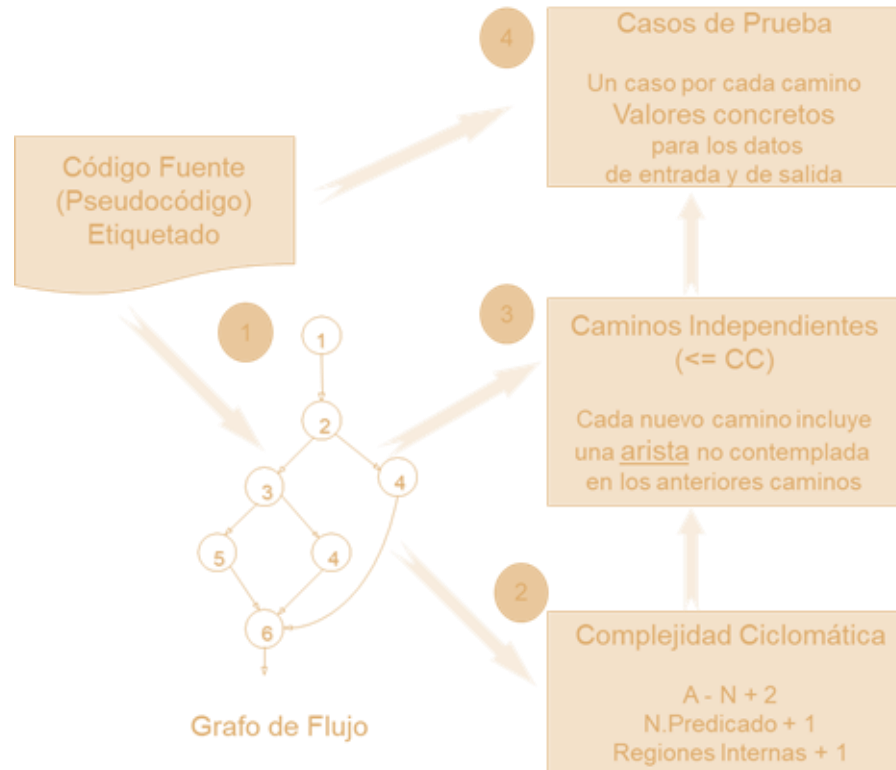
Prueba del camino básico: Derivación de casos de prueba

El método de prueba del camino básico se puede aplicar a un diseño procedural detallado (pseudocódigo) o al código fuente de la aplicación.

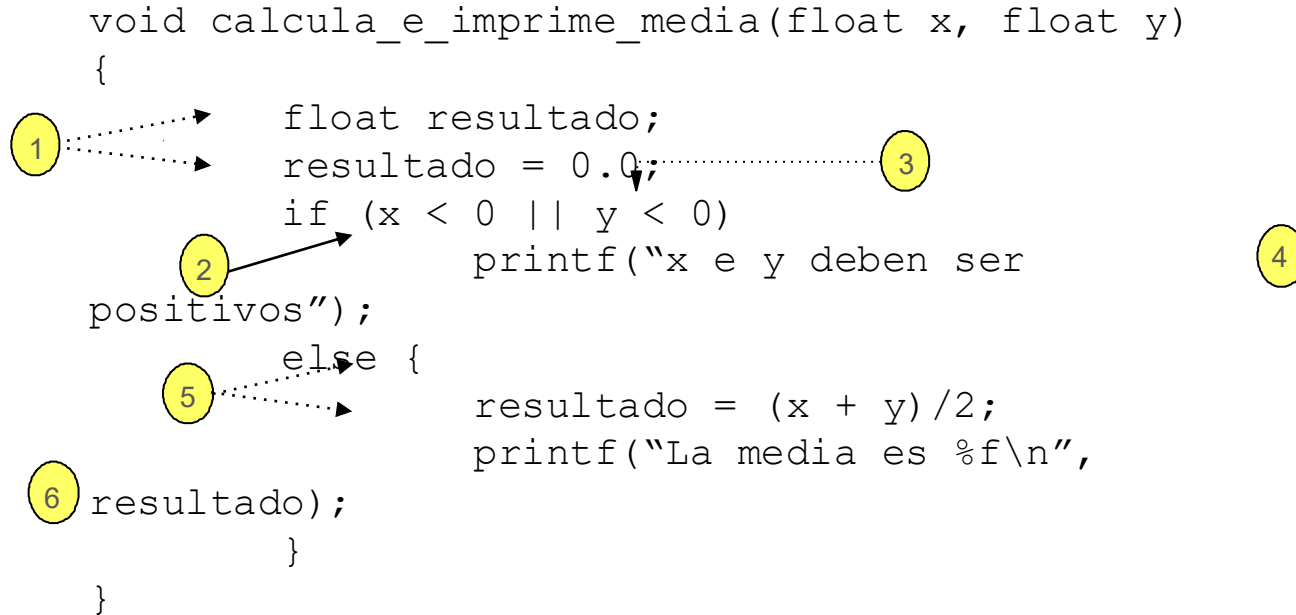
Pasos para diseñar los casos de prueba:

- **0.** Se etiqueta el código fuente, o el pseudocódigo, enumerando cada instrucción y cada condición simple.
- A partir del código fuente etiquetado, se dibuja el grafo de flujo asociado.
- Se calcula la complejidad ciclomática del grafo.
- Se determina un conjunto básico de caminos independientes.
- Se preparan los casos de prueba que obliguen a la ejecución de cada camino del conjunto básico.

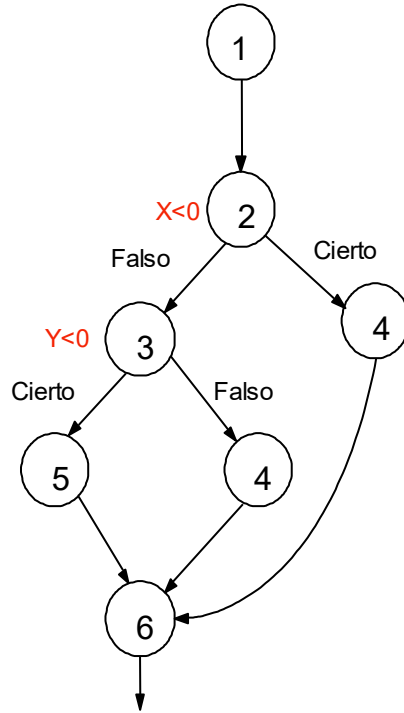
Prueba del camino básico: Derivación de casos de prueba



Prueba del camino básico: Ejemplo



Prueba del camino básico: Ejemplo



$V(G) = 3$ regiones. Por lo tanto, hay que determinar tres caminos independientes.

- Camino 1: 1-2-3-5-6
- Camino 2: 1-2-4-6
- Camino 3: 1-2-3-4-6

Casos de prueba para cada camino:

Camino 1: $x=3$, $y=5$, $rdo=4$

Camino 2: $x=-1$, $y=3$, $rdo=0$, error

Camino 3: $x=4$, $y=-3$, $rdo=0$, error

Pruebas de estructura de control

- La técnica de prueba del camino básico del punto anterior es una de las muchas existentes para las pruebas de la estructura de control.
- Aunque sea alta la efectividad de la prueba del camino básico, no nos asegura completamente la corrección del software.
- Veremos a continuación otras variantes de la prueba de la estructura de control. Estas variantes amplían el abanico de pruebas mejorando la fiabilidad de las pruebas de caja blanca.

Ejemplo:

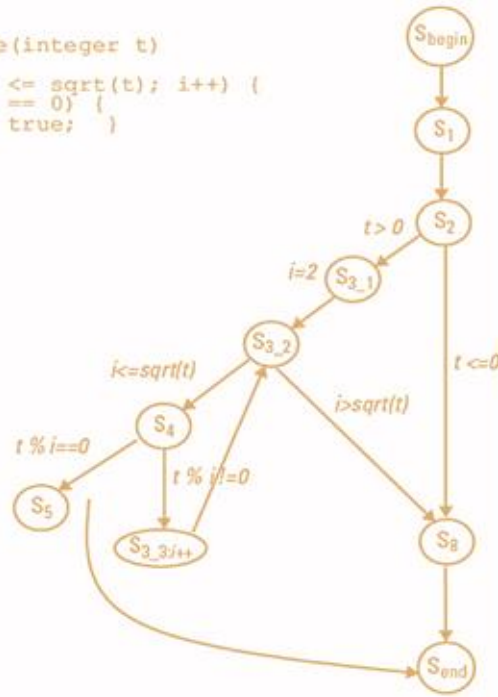
```
S1  Boolean Composite(integer t)
S2  if (t > 0) {
S3      for( i = 2; i <= sqrt(t); i++) {
S4          if ( t % i == 0) {
S5              return true;  }
S6      }
S7  }
S8  return false;
```

Ejemplo:

```

S1  Boolean Composite(integer t)
S2  if (t > 0) {
S3    for( i = 2; i <= sqrt(t); i++) {
S4      if ( t % i == 0) {
S5        return true; }
S6      }
S7    }
S8  return false;

```



Ejemplo:

```
public static boolean capicua(char[] list) {  
    int index = 0;  
    int l = list.length;  
    while (index < (l-1)) {  
        if (list[index] != list[(l-index)-1]) {  
            return false;  
        }  
        index++;  
    }  
    return true;  
}
```

Ejemplo Camino Básico: Código Fuente

```
int contar_letra(char cadena[10], char letra)
{
    int contador, n, lon;
    n=0; contador=0;
    lon = strlen (cadena);
    if (lon > 0) {
        do {
            if (cadena[contador] == letra) n++;
            contador++;
            lon--;
        } while (lon > 0);
    }
    return n;
}
```



Ejemplo Camino Básico: Código Fuente Etiquetado

```
int contar_letra(char cadena[10], char letra)
{
    int contador, n, lon;
    n=0; contador=0;
    lon = strlen (cadena);
    if (lon > 0) {
        do {
            if (cadena[contador] == letra)
                n++;
            contador++;
            lon--;
        } while (lon > 0);
    }
    return n;
}
```

1

2

3

4

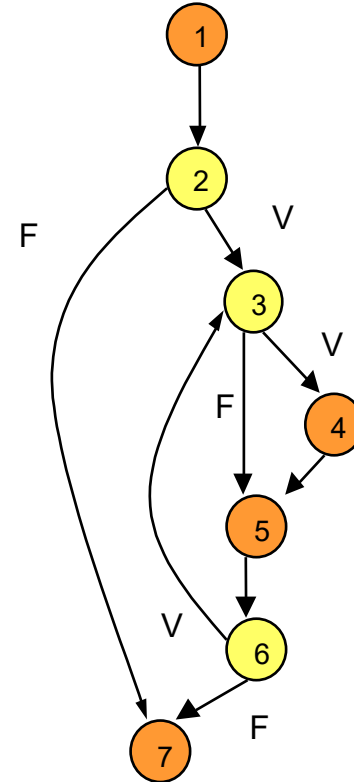
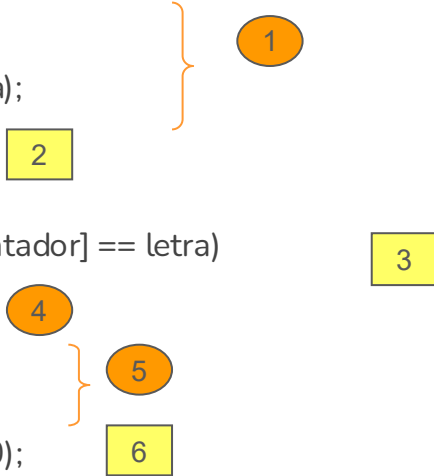
5

6

7

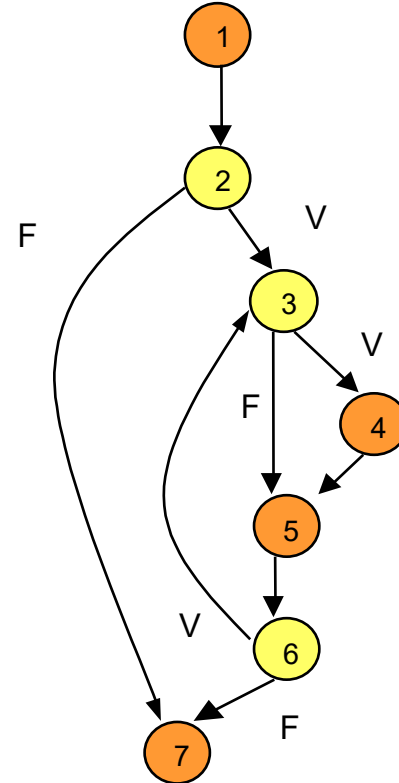
Ejemplo Camino Básico: Grafo de Flujo

```
int contar_letra(char cadena[10], char letra)
{
    int contador, n, lon;
    n=0; contador=0;
    lon = strlen (cadena);
    if (lon > 0) {
        do {
            if (cadena[contador] == letra)
                n++;
            contador++;
            lon--;
        } while (lon > 0);
    }
    return n;
}
```



Ejemplo Camino Básico: Caminos Independientes

- $V(G) = 4$;
Nodos=7; Aristas=9;
Nodos Predicado=3;
Regiones = 4
-
- Conjunto de caminos independientes:
 1. 1-2-7
 2. 1-2-3-4-5-6-7
 3. 1-2-3-5-6-7
 4. 1-2-3-4-5-6-3-5-6-7 (No es el único)



Ejemplo Camino Básico: Casos de Prueba

1.	1-2-7	cadena = ""	letra = 'a'	n = 0;
2.	1-2-3-4-5-6-7	cadena = "a"	letra = 'a'	n = 1;
3.	1-2-3-5-6-7	cadena = "b"	letra = 'a'	n = 0;
4.	1-2-3-4-5-6-3-5-6-7	cadena = "ab"	letra = 'a'	n = 1;

Pruebas de Caja Negra

La prueba de caja negra o prueba funcional principalmente analiza la compatibilidad en cuanto a las interfaces de cada uno de los componentes software entre sí.



Partición equivalente

- La partición equivalente es un método que divide el campo de entrada de un programa en clases de datos a partir de los cuales se pueden derivar casos de prueba.
- La partición equivalente define casos de prueba para descubrir clases de errores.
- Se define una condición de entrada para cada dato de entrada (valor numérico específico, rango de valores, conjunto de valores relacionados o condición lógica).

Partición equivalente

Las clases de equivalencia se pueden definir de acuerdo a las siguientes directrices:

- Si una condición de entrada especifica un rango, se define una clase de equivalencia válida y dos no válidas.
- Si la condición de entrada es un valor específico, se define una clase de equivalencia válida y dos no válidas.
- Si la condición de entrada especifica un miembro de un conjunto, se define una clase de equivalencia válida y otra no válida.
- Si la condición de entrada es lógica, se define una clase válida y otra no válida.

Ejemplo : Partición equivalente

Datos de entrada a una aplicación bancaria

Código de área: en blanco o número de 3 dígitos.

Prefijo: número de 3 dígitos que no comience por 0 o 1.

Sufijo: número de 4 dígitos.

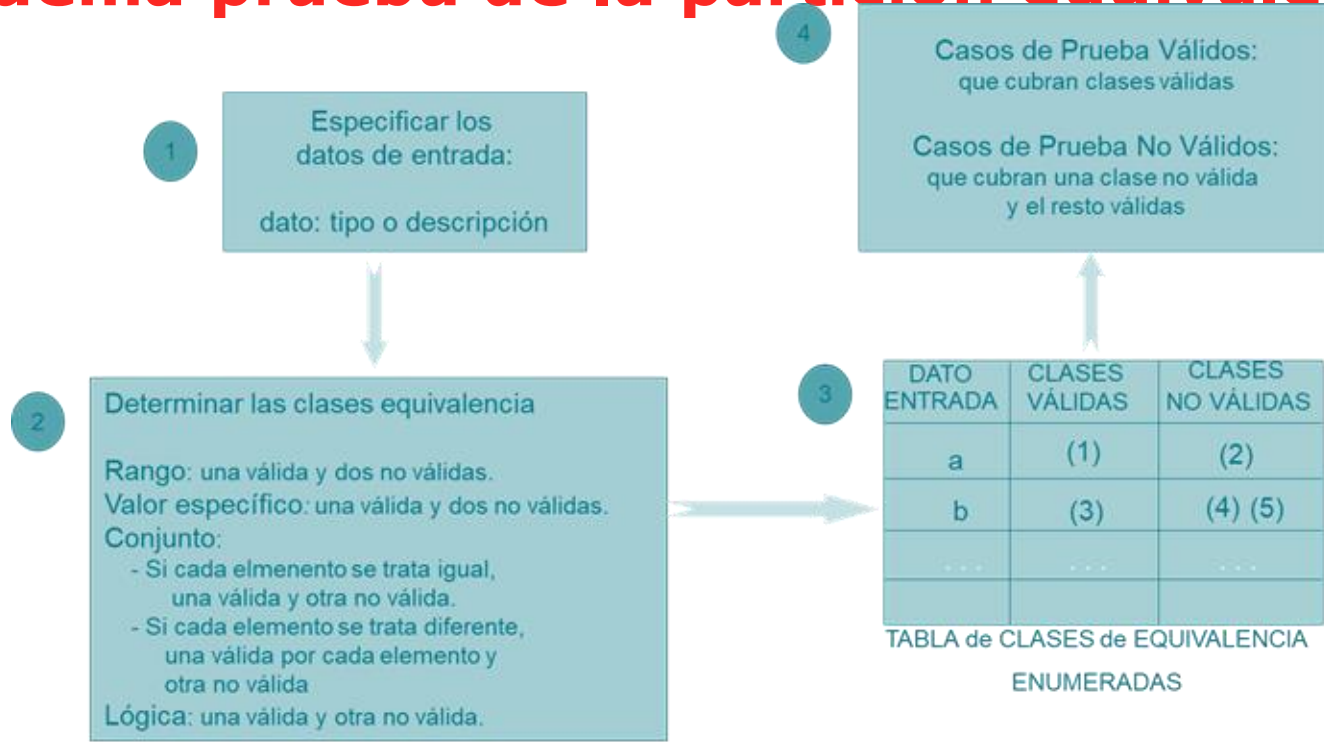
Contraseña: en blanco o valor alfanumérico de 6 caracteres.

Ordenes: “comprobar”, “depositar”, “pagar factura”, etc.

Ejemplo: Partición Equivalente

- Condiciones de entrada
 - Código de área: número de 3 dígitos o no es número
 - lógica: el código puede ser un número o no.
 - valor: número de 3 dígitos.
 - Prefijo: número de 3 dígitos que no comience por 0 o 1.
 - rango: valores entre 200 y 999.
 - Sufijo: número de 4 dígitos.
 - valor: número de 4 dígitos.
 - Contraseña: en blanco o valor alfanumérico de 6 caracteres.
 - lógica: la contraseña puede estar presente o no.
 - valor: cadena de 6 caracteres.
 - Ordenes: “comprobar”, “depositar”, “pagar factura”, etc.
 - conjunto: el de todas las ordenes posibles.

Esquema prueba de la partición equivalente



Ejemplo: Partición Equivalente

- Aplicación bancaria. Datos de entrada:
 - **Código de área:** número de 3 dígitos que no empieza por 0 ni por 1
 - **Nombre de identificación de operación:** 6 caracteres
 - **Órdenes posibles:** “cheque”, “depósito”, “pago factura”, “retirada de fondos”

Ejemplo Partición Equivalente

Determinar las clases de equivalencia

Código de área:

Lógica:

1 clase válida: número

Rango:

1 clase válida: $200 < \text{código} < 999$

2 clases no válidas: código < 200 ; código > 999

1 clase no válida: no es número

Nombre de identificación:

Valor específico:

1 clase válida: 6 caracteres

2 clases no válidas: más de 6 caracteres; menos de 6 caracteres

Órdenes posibles:

Conjunto de valores:

1 clase válida: 4 órdenes válidas

1 clase no válida: orden no válida



Ejemplo Partición Equivalente : Tabla de clases de equivalencia

Datos de Entrada	Clases Válidas	Clases No Válidas
Código de área	(1) $200 \leq \text{código} \leq 999$	(2) código < 200 (3) código > 999 (4) no es numérico
Identificación	(5) 6 caracteres	(6) menos de 6 caracteres (7) más de 6 caracteres
Orden	(8) "cheque" (9) "depósito" (10) "pago factura" (11) "retirada de fondos"	(12) ninguna orden válida

Ejemplo Partición Equivalente : Casos de Prueba Válidos

Código	Identificación	Orden	Clases Cubiertas
300	Nómina	"Depósito"	(1) ^C (5) ^C (9) ^C
400	Viajes	"Cheque"	(1) (5) (8) ^C
500	Coches	"Pago-factura"	(1) (5) (10) ^C
600	Comida	"Retirada-fondos"	(1) (5) (11) ^C

Ejemplo Partición Equivalente: Casos de Prueba NO Válidos

Código	Identificación	Orden	Clases Cubiertas
180	Viajes	"Pago-factura"	(2) ^C (5) (10)
1032	Nómina	"Depósito"	(3) ^C (5) (9)
XY	Compra	"Retirada-fondos"	(4) ^C (5) (11)
350	A	"Depósito"	(1) (6) ^C (9)
450	Regalos	"Cheque"	(1) (7) ^C (8)
550	Casa	&%4	(1) (5) (12) ^C

Análisis de valores límite

- La técnica de Análisis de Valores Límites selecciona casos de prueba que ejerciten los valores límite dada la tendencia de la aglomeración de errores en los extremos.
- Complementa la de la partición equivalente. En lugar de realizar la prueba con cualquier elemento de la partición equivalente, se escogen los valores en los bordes de la clase.
- Se derivan tanto casos de prueba a partir de las condiciones de entrada como con las de salida.

Otros tipos de prueba

- Recorridos (“walkthroughs”).
- Pruebas de robustez (“robustness testing”)
- Pruebas de aguante (“stress”)
- Prestaciones (“performance testing”)
- Conformidad u Homologación (“conformance testing”)
- Interoperabilidad (“interoperability testing”)
- Regresión (“regression testing”)
- Prueba de comparación

Herramientas automáticas de prueba

- **Analizadores estáticos.** Estos sistemas de análisis de programas soportan pruebas de las sentencias que consideran más débiles dentro de un programa.
- **Auditores de código.** Son filtros con el propósito de verificar que el código fuente cumple determinados criterios de calidad (dependerá fuertemente del lenguaje en cuestión).
- **Generadores de archivos de prueba.** Estos programas confeccionan automáticamente ficheros con datos que servirán de entrada a las aplicaciones.
- **Generadores de datos de prueba.** Confeccionan datos de entrada determinados que conlleven un comportamiento concreto del software.
- **Controladores de prueba.** Confeccionan y alimentan los datos de entrada y simulan el comportamiento de otros módulos para restringir el alcance de la prueba.

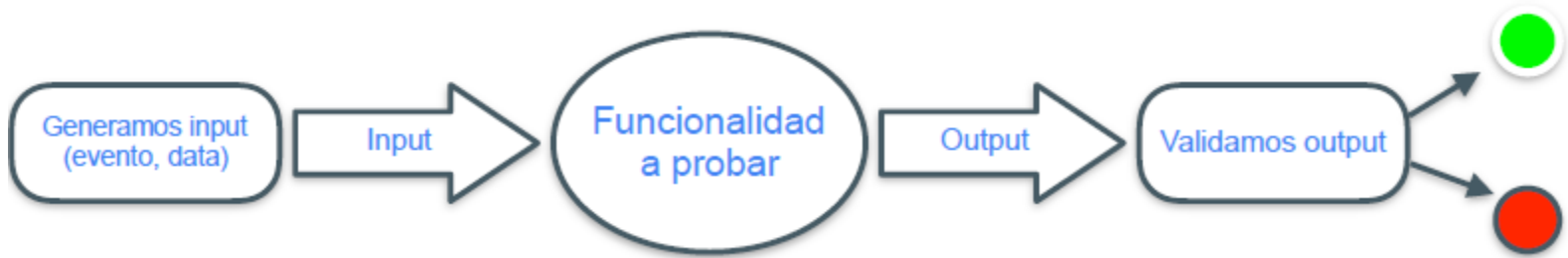
Automatización de pruebas unitarias.

La automatización de pruebas unitarias consiste en escribir y ejecutar pruebas de manera automática, sin intervención manual, para verificar que las unidades más pequeñas del código (como funciones, métodos o clases) funcionen correctamente. Este proceso es fundamental en el desarrollo de software moderno, ya que permite detectar errores rápidamente, mejorar la calidad del código y reducir el tiempo de prueba.

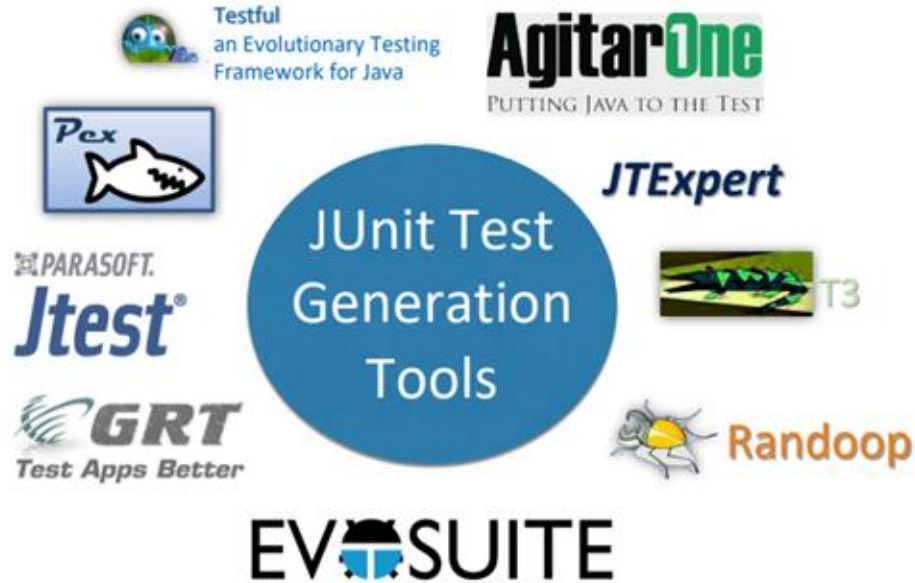


Test Automáticos

- La idea es poder simular la interacción de un usuario con el software implementado.
- Sigue el flujo básico de cualquier prueba de test automático:



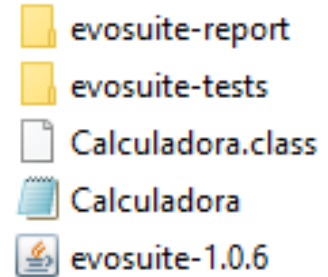
Generación de pruebas automáticas



Generación de pruebas automáticas

```
public class Calculadora{  
    public double suma(double a, double b){  
        return a+b;  
    }  
    public double divide(double a, double b){  
        if (b==0){  
            throw new ArithmeticException("No es posible la division entre cero");  
        }  
        return a/b;  
    }  
}
```

```
java -jar evosuite-1.0.6.jar -projectCP ./ -class Calculadora
```



Generación de pruebas automáticas

```
public class Calculadora_ESTest extends Calculadora_ESTest_scaffolding {
```

```
    @Test(timeout = 4000)
    public void testSumaReturningZero() throws Throwable {
        Calculadora calculadora0 = new Calculadora();
        double double0 = calculadora0.suma(0.0, 0.0);
        assertEquals(0.0, double0, 0.01);
    }
```

```
    @Test(timeout = 4000)
    public void testSumaReturningPositive() throws Throwable {
        Calculadora calculadora0 = new Calculadora();
        double double0 = calculadora0.suma(0.0, 2366.63002468);
        assertEquals(2366.63002468, double0, 0.01);
    }
```

```
java -jar evosuite-1.0.6.jar -projectCP ./ -class Calculadora -Dtest_naming_strategy=COVERAGE
```



Automatización de pruebas unitarias.

Beneficios de la Automatización de Pruebas Unitarias

- **Detección temprana de errores:** Las pruebas automatizadas se ejecutan constantemente, lo que permite identificar problemas en etapas iniciales del desarrollo.
- **Ahorro de tiempo y esfuerzo:** Las pruebas se ejecutan automáticamente, eliminando la necesidad de realizar pruebas manuales repetitivas.
- **Consistencia:** Las pruebas automatizadas garantizan que se ejecuten de la misma manera cada vez, reduciendo errores humanos.
- **Facilita el mantenimiento:** Permite verificar rápidamente que los cambios en el código no rompan funcionalidades existentes (regresión).
- **Integración continua:** Se pueden integrar con herramientas de CI/CD (como Jenkins, GitHub Actions o GitLab CI/CD) para ejecutar pruebas automáticamente con cada cambio en el código.



Herramientas para Automatización de Pruebas Unitarias

Existen diversas herramientas y frameworks que facilitan la automatización de pruebas unitarias. Algunas de las más populares son:

Por Lenguaje de Programación:

- **Java:** JUnit, TestNG.
- **Python:** pytest, unittest.
- **JavaScript:** Jest, Mocha, Jasmine.
- **C# (.NET):** NUnit, xUnit.net.
- **PHP:** PHPUnit.
- **C++:** Google Test (gTest), CppUnit.

Herramientas de Integración Continua:

- **Jenkins:** Permite ejecutar pruebas automáticamente en cada cambio del código.
- **GitHub Actions:** Automatiza la ejecución de pruebas en repositorios de GitHub.
- **GitLab CI/CD:** Ejecuta pruebas como parte de pipelines de integración y despliegue continuo.
- **CircleCI y Travis CI:** Plataformas de CI/CD que soportan la ejecución de pruebas automatizadas.

Herramientas de Cobertura de Código:

- **JaCoCo** (Java): Mide la cobertura de código durante las pruebas.
- **Coverage.py** (Python): Genera reportes de cobertura.
- **Istanbul** (JavaScript): Herramienta para medir la cobertura de pruebas en proyectos de JavaScript.

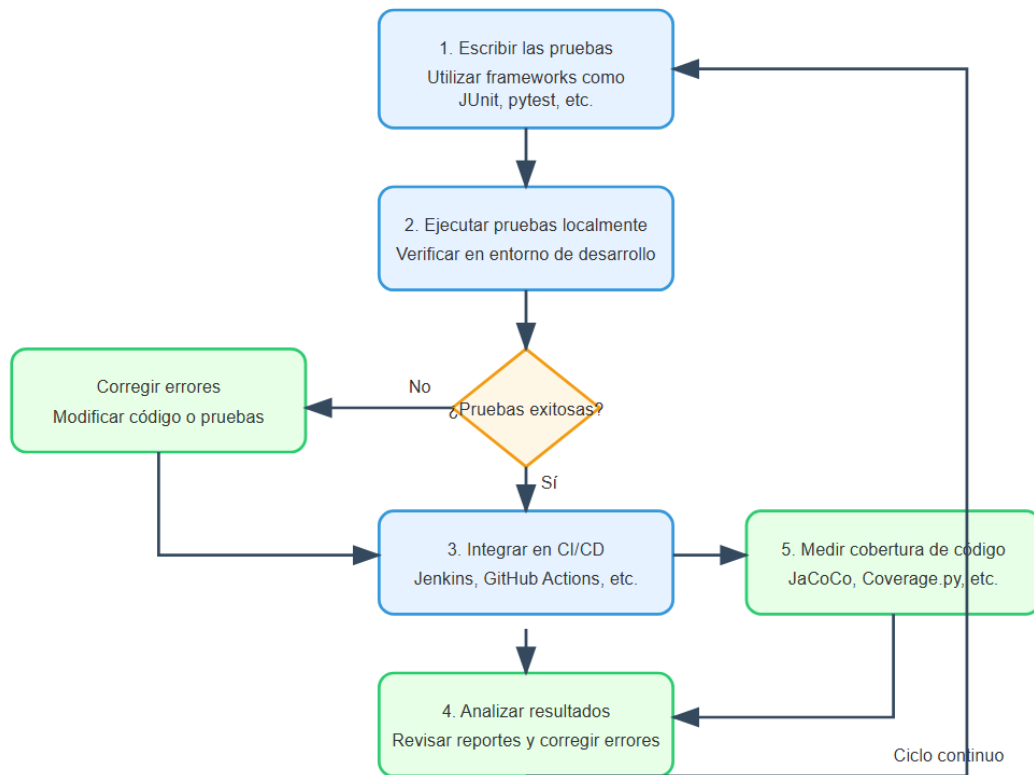


Proceso de Automatización de Pruebas Unitarias

- **Escribir las pruebas:**
 - Utilizar un framework de pruebas unitarias (como JUnit, pytest, etc.).
 - Diseñar casos de prueba que cubran las funcionalidades principales, casos límite y posibles errores.
- **Ejecutar las pruebas localmente:**
 - Verificar que las pruebas se ejecuten correctamente en el entorno de desarrollo.
- **Integrar las pruebas en un pipeline de CI/CD:**
 - Configurar herramientas como Jenkins, GitHub Actions o GitLab CI/CD para ejecutar las pruebas automáticamente con cada cambio en el código.
- **Analizar los resultados:**
 - Revisar los reportes generados por el framework de pruebas o herramientas de CI/CD.
 - Corregir los errores detectados y volver a ejecutar las pruebas.
- **Medir la cobertura de código:**
 - Utilizar herramientas de cobertura para asegurarse de que las pruebas cubran la mayor parte del código posible.



Proceso de Automatización de Pruebas Unitarias



Ejercicio de Estudio



Ejercicio 1 :

Desarrollar pruebas unitarias para una clase Calculator que implementa operaciones matemáticas básicas.

Requisitos:

1. Implementar pruebas unitarias utilizando JUnit 4 para la clase Calculator.
2. Verificar el correcto funcionamiento de las operaciones: suma, resta, multiplicación y división.
3. Incluir pruebas para casos límite y excepciones.
4. Lograr una cobertura de código de al menos 90%.

```
public class Calculator {

    public int add(int a, int b) {
        return a + b;
    }

    public int subtract(int a, int b) {
        return a - b;
    }

    public int multiply(int a, int b) {
        return a * b;
    }

    public int divide(int a, int b) {
        if (b == 0) {
            throw new ArithmeticException("División por cero no permitida");
        }
        return a / b;
    }

    public int gcd(int a, int b) {
        if (a < 0) a = -a;
        if (b < 0) b = -b;

        while (b != 0) {
            int temp = b;
            b = a % b;
            a = temp;
        }
        return a;
    }
}
```

```
public boolean isPrime(int n) {
    if (n < 2) {
        throw new IllegalArgumentException("Número debe ser mayor o igual a 2");
    }

    if (n == 2) return true;
    if (n % 2 == 0) return false;

    for (int i = 3; i <= Math.sqrt(n); i += 2) {
        if (n % i == 0) {
            return false;
        }
    }
    return true;
}
```



Solución:

```
1  import org.junit.Before;
2  import org.junit.Test;
3  import static org.junit.Assert.*;
4
5
6  public class CalculatorTest {
7
8      private Calculator calculator;
9
10     /**
11      * Configuración inicial antes de cada prueba.
12      */
13     @Before
14     public void setUp() {
15         calculator = new Calculator();
16     }
17
18     /**
19      * Prueba del método add con números positivos.
20      */
21     @Test
22     public void testAddPositiveNumbers() {
23         assertEquals(5, calculator.add(2, 3));
24         assertEquals(10, calculator.add(7, 3));
25     }
26
27     /**
28      * Prueba del método add con números negativos.
29      */
30     @Test
31     public void testAddNegativeNumbers() {
32         assertEquals(-5, calculator.add(-2, -3));
33         assertEquals(-1, calculator.add(-4, 3));
34     }
35 }
```

```
36
37     /**
38      * Prueba del método add con el valor máximo de int.
39      */
40     @Test
41     public void testAddWithMaxValue() {
42         assertEquals(Integer.MAX_VALUE, calculator.add(Integer.MAX_VALUE, 0));
43     }
44
45     /**
46      * Prueba del método subtract con números positivos.
47      */
48     @Test
49     public void testSubtractPositiveNumbers() {
50         assertEquals(2, calculator.subtract(5, 3));
51         assertEquals(-2, calculator.subtract(3, 5));
52     }
53
54     /**
55      * Prueba del método subtract con números negativos.
56      */
57     @Test
58     public void testSubtractNegativeNumbers() {
59         assertEquals(-2, calculator.subtract(-5, -3));
60         assertEquals(-8, calculator.subtract(-5, 3));
61     }
62
63     /**
64      * Prueba del método multiply con números positivos.
65      */
66     @Test
67     public void testMultiplyPositiveNumbers() {
68         assertEquals(15, calculator.multiply(5, 3));
69         assertEquals(0, calculator.multiply(0, 5));
70     }
71
72 }
```

Solución:

```
73  /**
74   * Prueba del método multiply con números negativos.
75   */
76  @Test
77  public void testMultiplyNegativeNumbers() {
78      assertEquals(-15, calculator.multiply(-5, 3));
79      assertEquals(15, calculator.multiply(-5, -3));
80  }
81
82  /**
83   * Prueba del método divide con números positivos.
84   */
85  @Test
86  public void testDividePositiveNumbers() {
87      assertEquals(2, calculator.divide(6, 3));
88      assertEquals(0, calculator.divide(3, 4));
89  }
90
91  /**
92   * Prueba del método divide con números negativos.
93   */
94  @Test
95  public void testDivideNegativeNumbers() {
96      assertEquals(-2, calculator.divide(-6, 3));
97      assertEquals(2, calculator.divide(-6, -3));
98  }
99
100 /**
101  * Prueba del método divide con divisor cero.
102  * Se espera una excepción ArithmeticException.
103  */
104 @Test(expected = ArithmeticException.class)
105 public void testDivideByZero() {
106     calculator.divide(5, 0);
107 }
108
```

```
109 /**
110  * Prueba del método gcd con números positivos.
111  */
112 @Test
113 public void testGcdPositiveNumbers() {
114     assertEquals(6, calculator.gcd(12, 18));
115     assertEquals(1, calculator.gcd(7, 13));
116     assertEquals(5, calculator.gcd(5, 0));
117 }
118
119 /**
120  * Prueba del método gcd con números negativos.
121  */
122 @Test
123 public void testGcdNegativeNumbers() {
124     assertEquals(6, calculator.gcd(-12, 18));
125     assertEquals(6, calculator.gcd(12, -18));
126     assertEquals(6, calculator.gcd(-12, -18));
127 }
128
129 /**
130  * Prueba del método isPrime con números primos.
131  */
132 @Test
133 public void testIsPrimeWithPrimeNumbers() {
134     assertTrue(calculator.isPrime(2));
135     assertTrue(calculator.isPrime(3));
136     assertTrue(calculator.isPrime(5));
137     assertTrue(calculator.isPrime(7));
138     assertTrue(calculator.isPrime(11));
139     assertTrue(calculator.isPrime(13));
140     assertTrue(calculator.isPrime(17));
141     assertTrue(calculator.isPrime(19));
142     assertTrue(calculator.isPrime(23));
143 }
144
```

Solución:

- **Estructura de las pruebas:** Se utiliza @Before para inicializar la calculadora antes de cada prueba. Cada método de la calculadora tiene múltiples pruebas para diferentes escenarios.
- **Cobertura de casos:**
 - **Casos normales:** Pruebas con valores típicos (números positivos y negativos).
 - **Casos límite:** Pruebas con valores extremos (Integer.MAX_VALUE, cero).
 - **Excepciones:** Pruebas para verificar que se lancen las excepciones esperadas.
- **Técnicas utilizadas:**
 - **Anotaciones JUnit:** @Test, @Before, @Test(expected=...)
 - **Métodos de aserción:** assertEquals, assertTrue, assertFalse
 - **Pruebas de excepciones:** Verificación de que se lancen las excepciones esperadas.
- **Documentación:**
 - Cada prueba está documentada con comentarios que explican su propósito.
 - Se sigue una convención de nomenclatura clara para los métodos de prueba.

```
145     /**
146      * Prueba del método isPrime con números no primos.
147      */
148     @Test
149     public void testIsPrimeWithNonPrimeNumbers() {
150         assertFalse(calculator.isPrime(4));
151         assertFalse(calculator.isPrime(6));
152         assertFalse(calculator.isPrime(8));
153         assertFalse(calculator.isPrime(9));
154         assertFalse(calculator.isPrime(10));
155         assertFalse(calculator.isPrime(12));
156         assertFalse(calculator.isPrime(15));
157     }
158
159     /**
160      * Prueba del método isPrime con números grandes.
161      */
162     @Test
163     public void testIsPrimeWithLargeNumbers() {
164         assertTrue(calculator.isPrime(997)); // Primo
165         assertFalse(calculator.isPrime(1000)); // No primo
166     }
167
168     /**
169      * Prueba del método isPrime con entrada inválida.
170      * Se espera una excepción IllegalArgumentException.
171      */
172     @Test(expected = IllegalArgumentException.class)
173     public void testIsPrimeWithInvalidInput() {
174         calculator.isPrime(1);
175     }
176
177     /**
178      * Prueba del método isPrime con entrada negativa.
179      * Se espera una excepción IllegalArgumentException.
180      */
181     @Test(expected = IllegalArgumentException.class)
182     public void testIsPrimeWithNegativeInput() {
183         calculator.isPrime(-5);
184     }
185 }
```



Ejercicio 2 :

Determine la complejidad ciclomatica del código proporcionado, construya el grafo asociado que represente el flujo de control del programa y, a partir de este, identifique los caminos independientes necesarios para realizar pruebas de caja blanca.

Asegúrese de calcular correctamente la complejidad ciclomatica utilizando las fórmulas adecuadas y de enumerar los caminos independientes que permitan cubrir todas las posibles rutas de ejecución del código.

```
Import java.io.*;
```

```
Public class Maximo
```

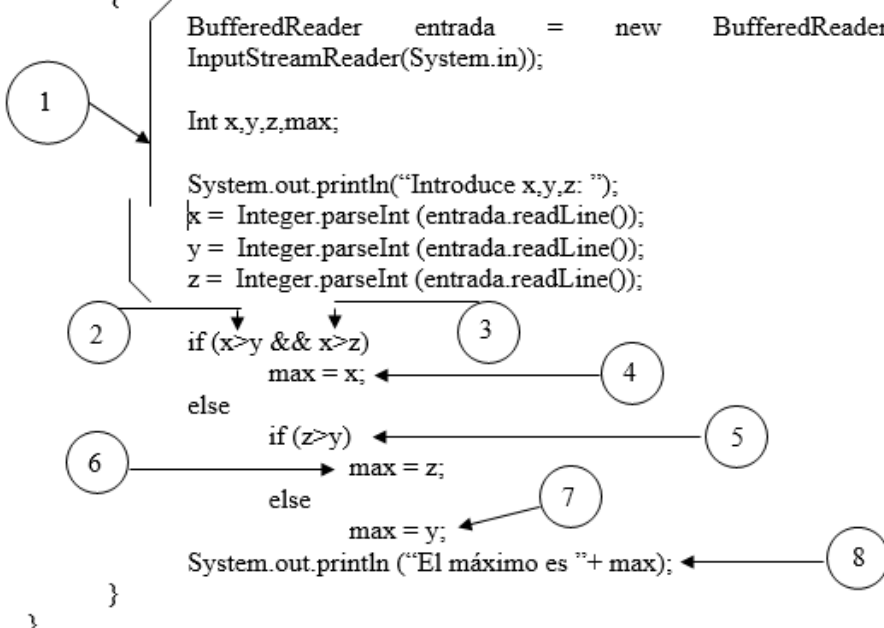
```
{  
    public static void main (String args[]) throws IOException  
    {  
        BufferedReader  entrada    =    new    BufferedReader    (new  
        InputStreamReader(System.in));  
  
        Int x,y,z,max;  
  
        System.out.println("Introduce x,y,z: ");  
        x = Integer.parseInt (entrada.readLine());  
        y = Integer.parseInt (entrada.readLine());  
        z = Integer.parseInt (entrada.readLine());  
  
        if (x>y && x>z)  
            max = x;  
        else  
            if (z>y)  
                max = z;  
            else  
                max = y;  
        System.out.println ("El máximo es "+ max);  
    }  
}
```

Solución:

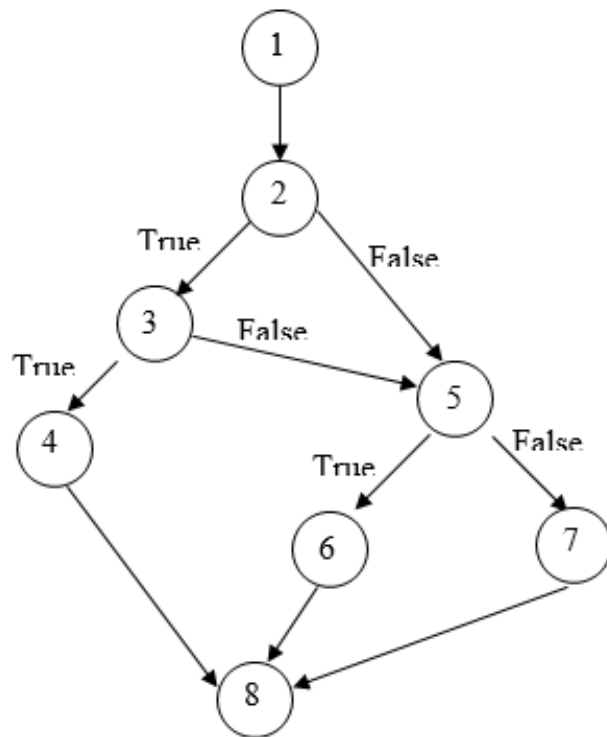
```
Import java.io.*;
```

```
Public class Maximo
```

```
{  
    public static void main (String args[]) throws IOException  
    {  
        1 BufferedReader  entrada  =  new  BufferedReader  (new  
        InputStreamReader(System.in));  
  
        Int x,y,z,max;  
  
        System.out.println("Introduce x,y,z: ");  
        x = Integer.parseInt (entrada.readLine());  
        y = Integer.parseInt (entrada.readLine());  
        z = Integer.parseInt (entrada.readLine());  
  
        2 if (x>y && x>z) 3  
            max = x; 4  
        else  
            if (z>y) 5  
                6 max = z;  
            else 7  
                max = y;  
        8 System.out.println ("El máximo es "+ max);  
    }  
}
```



Solución:



Solución:

Calculamos la complejidad ciclomática de McCabe :

$$V(G) = a - n + 2 = 10 - 8 + 2 = 4$$

$$V(G) = r = 4$$

$$V(G) = c + 1 = 3 + 1 = 4$$

Por lo tanto tendremos cuatro caminos independientes, que mirando el grafo de flujo deducimos serán los siguientes:

- Camino 1 ➔ 1 - 2 - 3 - 4 - 8
- Camino 2 ➔ 1 - 2 - 3 - 5 - 6 - 8
- Camino 3 ➔ 1 - 2 - 5 - 6 - 8
- Camino 4 ➔ 1 - 2 - 5 - 7 - 8

Solución:

Camino	Características	Caso de Prueba		
		x	y	z
Camino 1	$x > y, x > z$	10	3	3
Camino 2	$y < x < z$	5	2	10
Camino 3	$x < y < z$	2	5	8
Camino 4	$x < y, z < y$	5	10	5

Ejercicio 3 :

Dada la siguiente especificación:

Un programa toma como entrada un fichero cuyo formato de registro es el siguiente:

Numero-empleado	Nombre-empleado	Meses-Trabajo	Directivo
-----------------	-----------------	---------------	-----------

donde:

- Numero-empleado es un campo de números enteros positivos de 3 dígitos (excluido el 000).
- Nombre-empleado es un campo alfanumérico de 10 caracteres.
- Meses-Trabajo es un campo que indica el número de meses que lleva trabajando el empleado; es un entero positivo (incluye el 000) de 3 dígitos.
- Directivo es un campo de un solo carácter que puede ser «+» para indicar que el empleado es un directivo y «-» para indicar que no lo es.

Crear una Tabla de Clases de Equivalencia y genere los Casos de Prueba

Solución:

Campo: Numero-empleado:

- **Condición de entrada:** Especifica un rango (001-999).
- **Clases de equivalencia:**
 - **Válida:** Números enteros positivos de 3 dígitos entre 001 y 999 (inclusive).
 - **No válida 1:** Números menores al rango (e.g., 000 o valores negativos como -123).
 - **No válida 2:** Números mayores al rango (e.g., 1000 o más).

Solución:

Campo: Nombre-empleado

- **Condición de entrada:** Especifica un valor específico (cadena alfanumérica de exactamente 10 caracteres).
- **Clases de equivalencia:**
 - **Válida:** Cadenas alfanuméricas de exactamente 10 caracteres (e.g., "JuanCarlos").
 - **No válida 1:** Cadenas con menos de 10 caracteres (e.g., "Juan").
 - **No válida 2:** Cadenas con más de 10 caracteres o con caracteres no alfanuméricos (e.g., "JuanCarlos12" o "Juan@12345").

Solución:

Campo: Meses-Trabajo:

- **Condición de entrada:** Especifica un rango (000-999).
- **Clases de equivalencia:**
 - **Válida:** Números enteros positivos de 3 dígitos entre 000 y 999 (inclusive).
 - **No válida 1:** Números menores al rango (e.g., valores negativos como -12).
 - **No válida 2:** Números mayores al rango (e.g., 1000 o más).

Solución:

Campo: Directivo:

- **Condición de entrada:** Especifica un miembro de un conjunto (carácter «+» o «-»).
- **Clases de equivalencia:**
 - **Válida:** Carácter «+» o «-».
 - **No válida:** Cualquier carácter distinto de «+» o «-» (e.g., «*», «A», «1») o una cadena vacía.

Solución:

Campo	Condición de Entrada	Clases Válidas	Clases No Válidas
Numero-empleado	Rango (001-999)	Números enteros positivos de 3 dígitos entre 001 y 999	- Números menores a 001 (e.g., 000, -123) - Números mayores a 999 (e.g., 1000)
Nombre-empleado	Valor específico (cadena de 10 caracteres)	Cadenas alfanuméricas de exactamente 10 caracteres	- Cadenas con menos de 10 caracteres (e.g., "Juan") - Cadenas con más de 10 caracteres o caracteres no alfanuméricos (e.g., "JuanCarlos12", "Juan@12345")
Meses-Trabajo	Rango (000-999)	Números enteros positivos de 3 dígitos entre 000 y 999	- Números menores a 000 (e.g., -12) - Números mayores a 999 (e.g., 1000)
Directivo	Miembro de un conjunto («+» o «-»)	Carácter «+» o «-»	- Cualquier carácter distinto de «+» o «-» (e.g., «*», «A», «1») - Cadena vacía

Solución:

Caso de Prueba	Numero-empleado	Nombre-empleado	Meses-Trabajo	Directivo	Resultado Esperado
1	123	JuanCarlos	012	+	Registro válido
2	001	AnaMaria12	000	-	Registro válido
3	999	Pedro123456	999	+	Registro válido
4	000	JuanCarlos	012	+	Error: Número de empleado inválido
5	1000	JuanCarlos	012	+	Error: Número de empleado inválido
6	-123	JuanCarlos	012	+	Error: Número de empleado inválido
7	123	Juan	012	+	Error: Nombre de empleado inválido
8	123	JuanCarlos123	012	+	Error: Nombre de empleado inválido
9	123	Juan@12345	012	+	Error: Nombre de empleado inválido
10	123	JuanCarlos	1000	+	Error: Meses de trabajo inválido
11	123	JuanCarlos	-12	+	Error: Meses de trabajo inválido
12	123	JuanCarlos	12	+	Error: Meses de trabajo inválido
13	123	JuanCarlos	012	*	Error: Directivo inválido
14	123	JuanCarlos	012	A	Error: Directivo inválido
15	123	JuanCarlos	012		Error: Directivo inválido

¿Qué se logró?



- ✓ Diseñar, implementar y ejecutar pruebas unitarias, diferenciando entre pruebas estáticas y dinámicas, y aplicando técnicas de caja blanca y caja negra para garantizar la calidad del código.
- ✓ Utilizar herramientas específicas para pruebas unitarias, como JUnit e integrar procesos de automatización que optimizan el tiempo y la eficiencia en el desarrollo de software.
- ✓ Comprender cómo las pruebas unitarias contribuyen a la detección temprana de errores, la mejora de la mantenibilidad del código y la entrega de software confiable y robusto.

¿Qué debemos hacer para alcanzar el objetivo del tema?



- ✓ Repasar la teoría, los ejemplos y casos resueltos.



Referencias Bibliográficas

- Proceso de Construcción de Software 2, Mag. Natalí Flores Lafosse, Maestría en Informática de la PUCP, 2018.
- Sommerville, I. (2005). Ingeniería del software. Pearson educación.
- Bourque, P., & Fairley, R. E. (2014). Guide to the software engineering body of knowledge (SWEBOK (R)): Version 3.0. IEEE Computer Society Press.

GRACIAS